Here are comprehensive answers to your PHP questions, including code examples and detailed explanations.

Part-1: Core PHP Basics & Forms

Q1. echo vs print (Concept + application)

PHP Script:

code PHP
downloadcontent_copy
expand_less

```
<?php
// Using echo to display a welcome message
echo "<h1>Welcome to PHP with echo!</h1>";
echo "<p>This is a paragraph using echo.</p>";

// Using print to display another welcome message
print "<h2>Hello from PHP with print!</h2>";
print "<p>Another paragraph, this time with print.</p>";
?>
```

Explanation of Differences:

1. **Return Value:**
    - echo has no return value. It is a language construct that outputs one or more strings. Because it doesn't return anything, it's slightly faster.
    - print has a return value of 1 (true) for success. This means print can be used in expressions, which echo cannot.
    - **Application Example for Return Value (print only):**

      code PHP
      downloadcontent_copy
      expand_less

      ```
      <?php
      $result = print "This message will be printed. ";
      echo "The print function returned: " . $result; // Output: The print
      function returned: 1
      ?>
      ```

2. **Performance or Usage Context:**
    - **Performance:** echo is marginally faster than print because it doesn't calculate a return value. For simple string output, this difference is negligible, but it can be a consideration in high-performance applications with extensive output.
    - **Usage Context:**
        - echo can take multiple arguments (separated by commas) to output multiple strings efficiently, like echo "Hello", " World";.
        - print can only take one argument.

- print can be useful when you need to know if the output operation was successful (e.g., in a conditional statement), though this is rarely needed for basic output. echo is generally preferred for its flexibility with multiple arguments and slightly better performance for pure output tasks.

Q2. Superglobal variables

PHP Form (index.html or index.php for the form part):

code Html
downloadcontent_copy
expand_less

```
<!DOCTYPE html>
<html>
<head>
    <title>Username Form</title>
</head>
<body>
    <form action="display_username.php" method="POST">
        <label for="username">Username:</label><br>
        <input type="text" id="username" name="username"><br><br>
        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

display_username.php:

code PHP
downloadcontent_copy
expand_less

```
<?php
// Access the submitted username using the $_POST superglobal variable
if (isset($_POST['username'])) {
    $username = $_POST['username'];
    echo "<h1>Welcome, " . htmlspecialchars($username) . "!</h1>";
} else {
    echo "<p>No username submitted.</p>";
}
?>
```

Explanation of Superglobal Variables:

A **superglobal variable** in PHP is a special type of built-in variable that is always accessible from anywhere in a script, regardless of the scope (i.e., you can access them inside or outside functions and classes without needing to declare them as global). This makes them very convenient for handling data that needs to be available throughout the application.

Two Examples of Superglobal Variables:

1. **$_POST**: An associative array of variables passed to the current script via the HTTP POST method. It's used to collect form data after submitting an HTML form with method="POST".

2. **$_GET**: An associative array of variables passed to the current script via the HTTP GET method. It's used to collect form data after submitting an HTML form with method="GET", or variables sent in the URL query string.
3. **$_SERVER**: Contains information about headers, paths, and script locations.
4. **$_SESSION**: An associative array containing session variables available to the current script.

Q3. Constants in PHP

PHP Program:

code PHP
downloadcontent_copy
expand_less

```
<?php
// Define a constant named SITE_NAME
define("SITE_NAME", "MyCollegePortal");

// Display the constant value in an HTML heading
echo "<h1>Welcome to " . SITE_NAME . "</h1>";

// Another way to define constants in PHP 5.6+ with const keyword (global scope
only)
const APP_VERSION = "1.0.0";
echo "<p>Application Version: " . APP_VERSION . "</p>";
?>
```

Explanation of How Constants Differ from Variables:

1. **Value Immutability:** Once a constant is defined, its value cannot be changed or undefined during script execution. Variables, on the other hand, can have their values reassigned any number of times.
2. **Naming Convention:** By convention, PHP constants are usually named in uppercase letters, often with underscores between words (e.g., SITE_NAME, MAX_USERS). Variables typically start with a lowercase letter or underscore (e.g., $username, $_count).
3. **Scope:** Constants are global by default. Once defined, they can be accessed from anywhere in the script without using the global keyword. Variables have specific scopes (local, global, static) and often require the global keyword to be accessed outside their defined scope (e.g., a global variable inside a function).
4. **Declaration:** Constants are defined using the define() function (or const keyword for class constants and global-scope constants since PHP 5.6). Variables are declared by assigning a value to them using a $ prefix.
5. **No $ Prefix:** Constants do not have a $ sign before their name when being used, unlike variables.

Q4. trim() function (Practical use)

PHP Script:

code PHP

downloadcontent_copy
expand_less
```
<?php
$raw_name = "   Ramesh   ";

echo "<p>Original name (with spaces): '" . $raw_name . "'</p>";

// Remove extra spaces using trim()
$cleaned_name = trim($raw_name);

echo "<p>Cleaned name (without spaces): '" . $cleaned_name . "'</p>";

// Demonstrating with other forms of whitespace
$another_raw_name = "\t John Doe \n"; // Includes tab and newline
$another_cleaned_name = trim($another_raw_name);
echo "<p>Another original name: '" . $another_raw_name . "'</p>";
echo "<p>Another cleaned name: '" . $another_cleaned_name . "'</p>";
?>
```

Explanation of the Purpose of trim() function:

The trim() function in PHP is used to remove whitespace or other predefined characters from both the beginning and end of a string. Whitespace characters include spaces, tabs (\t), newlines (\n), carriage returns (\r), null bytes (\0), and vertical tabs (\x0B).

Purpose:

- **Data Cleaning:** It's particularly useful for cleaning up user input from forms, where users might accidentally or intentionally add leading or trailing spaces. These extra spaces can cause issues with data validation, comparisons, or storage.
- **Consistency:** Ensures data consistency by standardizing strings. For example, " apple" and "apple " would be treated as different strings without trim(), but trim() would make them both "apple".
- **Preventing Errors:** Helps prevent errors in database queries, file paths, or other operations where exact string matches are crucial.

By default, trim() removes all common whitespace characters. You can also specify an optional second argument to define a list of characters you want to strip.

Q5. Static variable inside function

PHP Function counter():

code PHP
downloadcontent_copy
expand_less
```
<?php
function counter() {
    static $count = 0; // Declare a static variable and initialize it once
    $count++;         // Increment the static variable
    echo "<p>Counter value: " . $count . "</p>";
}

echo "First call to counter(): ";
counter();
```

```
echo "Second call to counter(): ";
counter();

echo "Third call to counter(): ";
counter();

echo "<hr>";

// Demonstrating that a non-static local variable would reset
function simple_counter() {
    $nonStaticCount = 0; // This variable resets on each call
    $nonStaticCount++;
    echo "<p>Non-Static Counter value: " . $nonStaticCount . "</p>";
}
echo "First call to simple_counter(): ";
simple_counter(); // Output: Non-Static Counter value: 1
echo "Second call to simple_counter(): ";
simple_counter(); // Output: Non-Static Counter value: 1
?>
```

Explanation of How Static Works:

When a local variable is declared within a function using the static keyword, it behaves differently from a regular local variable:

1. **Initialization:** A static variable is initialized only once, the very first time the function is called. In the counter() example, $count = 0; is executed only on the first invocation.
2. **Persistence:** Unlike regular local variables, whose values are destroyed once the function completes execution, a static variable's value persists between successive calls to the same function. When counter() is called again, $count retains its last updated value, rather than being re-initialized to 0.
3. **Scope:** Despite its persistence, the static variable still maintains its local scope. It cannot be accessed from outside the function where it was declared.

In the example:

- **First call:** $count is initialized to 0, then incremented to 1, and 1 is displayed.
- **Second call:** $count retains its value of 1 from the previous call. It's then incremented to 2, and 2 is displayed.
- **Third call:** $count retains its value of 2. It's then incremented to 3, and 3 is displayed.

This demonstrates how static variables allow a function to "remember" information between calls without making the variable global.

Q6. POST vs GET (Applied explanation)

Login Form (login.html or login.php for the form part):

code Html
play_circledownloadcontent_copy
expand_less

```
<!DOCTYPE html>
<html>
<head>
    <title>Login Form</title>
```

```
</head>
<body>
    <form action="process_login.php" method="POST">
        <label for="username">Username:</label><br>
        <input type="text" id="username" name="username"><br><br>
        <label for="password">Password:</label><br>
        <input type="password" id="password" name="password"><br><br>
        <input type="submit" value="Login">
    </form>
</body>
</html>
```

process_login.php:

code PHP

downloadcontent_copy

expand_less

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $username = htmlspecialchars($_POST['username']);
    $password = htmlspecialchars($_POST['password']); // In a real app, hash and
verify this!

    echo "<h1>Login Attempt:</h1>";
    echo "<p>Username: " . $username . "</p>";
    echo "<p>Password: " . $password . " (For demonstration. Never display real
passwords!)</p>";

    // In a real application, you would verify username and hashed password
against a database
    if ($username === "admin" && $password === "password123") {
        echo "<p style='color: green;'>Login successful!</p>";
    } else {
        echo "<p style='color: red;'>Invalid username or password.</p>";
    }

} else {
    echo "<p>This page should be accessed via a POST request from the login
form.</p>";
}
?>
```

Explanation of Two Advantages of POST over GET:

1. **Security/Privacy for Sensitive Data:**
    - **POST:** Data sent via POST is included in the body of the HTTP request. It is not appended to the URL and therefore not visible in the browser's address bar or browser history. This makes POST suitable for submitting sensitive information like passwords, credit card numbers, or other personal details, as it offers a basic level of privacy from casual observation.
    - **GET:** Data sent via GET is appended to the URL as query parameters. This means it's visible to anyone looking at the browser's address bar, can be bookmarked, appears in server logs, and is stored in browser history. This is a significant security risk for sensitive information.
2. **Larger Data Capacity:**

- **POST:** There is generally no practical limit to the amount of data that can be sent in the body of a POST request. While server configurations might impose limits (e.g., post_max_size in PHP.ini), these are typically much larger than GET's limits. This makes POST ideal for submitting large amounts of text, file uploads, or complex form data.
- **GET:** The amount of data that can be sent via GET is limited by the maximum length of a URL, which varies by browser and server but is typically around 2048 characters. Exceeding this limit will result in truncation or errors.

Q7. json_encode() purpose

PHP Script:

code PHP
downloadcontent_copy
expand_less

```
<?php
// Create an associative array for student data
$student_data = array(
    "name"        => "Alice Wonderland",
    "roll_number" => "CS101",
    "marks"       => array(
        "math"    => 85,
        "science" => 92,
        "english" => 78
    ),
    "is_enrolled" => true
);

echo "<h2>Original PHP Associative Array:</h2>";
echo "<pre>";
print_r($student_data);
echo "</pre>";

// Convert the associative array into JSON format
$json_output = json_encode($student_data, JSON_PRETTY_PRINT); //
JSON_PRETTY_PRINT makes it readable

echo "<h2>Converted JSON Data:</h2>";
echo "<pre>";
echo $json_output;
echo "</pre>";
?>
```

Explanation of the Primary Purpose of json_encode():

The primary purpose of the json_encode() function in PHP is to **convert PHP values (primarily arrays and objects) into a JSON (JavaScript Object Notation) formatted string.**

Why is this important?

- **Interoperability:** JSON is a language-independent data format widely used for exchanging data between different systems and programming languages (e.g., PHP backend with a JavaScript frontend, mobile applications, or APIs). json_encode() provides a standardized way for PHP to generate data that other systems can easily understand and parse.

- **API Responses:** When building web services or APIs with PHP, data is almost always returned in JSON format. json_encode() is essential for preparing these responses.
- **Client-Side Communication:** It facilitates sending data from a PHP server to a web browser's JavaScript. JavaScript can then easily parse and use this JSON data (since JSON is a subset of JavaScript object literal syntax).
- **Data Storage:** JSON strings can also be stored in databases (e.g., in a text field) or files, offering a flexible way to store structured data that can later be retrieved and decoded back into PHP.

In essence, json_encode() acts as a bridge, allowing PHP to speak the common data exchange language of the modern web.

Part-2: Variables, Arrays, Control, and File Upload

Q8. Variable scope (local, global, static)

PHP Program:

code PHP
downloadcontent_copy
expand_less

```
<?php
// Global variable declaration
$globalVar = "I am a global variable.";

function demonstrateScopes() {
    // Local variable declaration
    $localVar = "I am a local variable.";
    echo "<p>Inside function:</p>";
    echo "<p>Local variable: " . $localVar . "</p>"; // Accessible here

    // Attempting to access globalVar directly (will fail without 'global'
keyword)
    // echo "<p>Global variable (direct access - will be empty): " .
$globalVar . "</p>";

    // Accessing global variable using 'global' keyword
    global $globalVar;
    echo "<p>Global variable (accessed with 'global'): " . $globalVar . "</p>";

    // Accessing global variable using $GLOBALS superglobal
    echo "<p>Global variable (accessed with \$GLOBALS): " .
$GLOBALS['globalVar'] . "</p>";

    // Static variable
    static $staticVar = 0; // Initialized once
    $staticVar++;
    echo "<p>Static variable (incremented): " . $staticVar . "</p>";

    // Another local variable, just for this call
    $tempLocalVar = "This resets every time.";
    echo "<p>Temp Local variable: " . $tempLocalVar . "</p>";
}

echo "<h2>Demonstrating Variable Scopes</h2>";
```

```
echo "<h3>First call to demonstrateScopes():</h3>";
demonstrateScopes();
echo "<h3>Second call to demonstrateScopes():</h3>";
demonstrateScopes();

echo "<h3>Outside function:</h3>";
echo "<p>Global variable: " . $globalVar . "</p>"; // Accessible here

// Attempting to access local or static variables from outside (will cause
error/fail)
// echo "<p>Local variable (outside function - will fail): " . $localVar .
"</p>";
// echo "<p>Static variable (outside function - will fail): " . $staticVar .
"</p>";
?>
```

Explanation of How Each Scope Behaves:

1. **Local Scope:**
    - **Behavior:** A variable declared inside a function has local scope. It can only be accessed and modified within that specific function. Once the function finishes execution, the memory allocated for local variables is freed, and their values are destroyed.
    - **Demonstration:** In demonstrateScopes(), $localVar and $tempLocalVar are local. They are created and destroyed with each function call.
2. **Global Scope:**
    - **Behavior:** A variable declared outside any function has global scope. It can be accessed from anywhere in the script *except* directly inside a function. To access or modify a global variable from within a function, you must explicitly declare it as global inside the function using the global keyword, or use the GLOBALS superglobal array.
    - **Demonstration:** $globalVar is a global variable. It's accessible outside the function. Inside the function, we show two ways to access it (global $globalVar; and $GLOBALS['globalVar']).
3. **Static Scope:**
    - **Behavior:** A variable declared as static inside a function has a special property: it is initialized only once (the first time the function is called), but its value persists between subsequent calls to that function. Like local variables, static variables are still restricted to the function's scope and cannot be accessed from outside.
    - **Demonstration:** $staticVar inside demonstrateScopes() is initialized to 0 only on the first call. In subsequent calls, it retains its value and continues incrementing, proving its persistence.

Q9. File upload process

PHP Script (upload.php):

code PHP

downloadcontent_copy
expand_less

```
<!DOCTYPE html>
<html>
<head>
    <title>File Upload</title>
</head>
<body>
    <h1>Upload an Image</h1>
    <form action="upload.php" method="POST" enctype="multipart/form-data">
        Select image to upload:
        <input type="file" name="fileToUpload" id="fileToUpload">
        <input type="submit" value="Upload Image" name="submit">
    </form>

    <?php
    if (isset($_POST["submit"])) {
        $target_dir = "uploads/"; // Directory where the file will be placed
        $target_file = $target_dir . basename($_FILES["fileToUpload"]
["name"]); // Path of the file to be uploaded
        $uploadOk = 1; // Flag to check if upload is successful
        $imageFileType = strtolower(pathinfo($target_file, PATHINFO_EXTENSION));
// Get file extension

        // Create uploads directory if it doesn't exist
        if (!is_dir($target_dir)) {
            mkdir($target_dir, 0777, true);
        }

        // --- Security Consideration 1: File Type Validation ---
        // Allow certain file formats
        $allowed_extensions = array("jpg", "jpeg", "png", "gif");
        if (!in_array($imageFileType, $allowed_extensions)) {
            echo "<p style='color: red;'>Sorry, only JPG, JPEG, PNG & GIF files
are allowed.</p>";
            $uploadOk = 0;
        }

        // --- Security Consideration 2: File Size Validation ---
        // Check file size (e.g., max 500KB)
        if ($_FILES["fileToUpload"]["size"] > 500000) { // 500 KB
            echo "<p style='color: red;'>Sorry, your file is too large (max
500KB).</p>";
            $uploadOk = 0;
        }

        // Check if $uploadOk is set to 0 by an error
        if ($uploadOk == 0) {
            echo "<p style='color: red;'>Sorry, your file was not
uploaded.</p>";
        // if everything is ok, try to upload file
        } else {
            if (move_uploaded_file($_FILES["fileToUpload"]["tmp_name"],
$target_file)) {
                echo "<p style='color: green;'>The file " .
htmlspecialchars(basename($_FILES["fileToUpload"]["name"])) . " has been
uploaded.</p>";
                echo "<img src='" . $target_file . "' width='200px'
alt='Uploaded Image'>";
            } else {
                echo "<p style='color: red;'>Sorry, there was an error uploading
your file.</p>";
            }
```

```
        }
    }
    ?>
</body>
</html>
```

Explanation of File Upload Process and Security Considerations:

Steps in the Upload Process:

1. **HTML Form Setup:**
    - An HTML <form> is used with method="POST" and crucially, enctype="multipart/form-data". enctype tells the browser to encode the form data as a multipart message, allowing files to be sent.
    - An <input type="file" name="fileToUpload"> element allows the user to select a file from their local system.
2. **Server-Side Script (upload.php):**
    - When the form is submitted, PHP automatically populates the $_FILES superglobal array with information about the uploaded file(s).
    - $_FILES["fileToUpload"] (where "fileToUpload" is the name attribute of the input field) will contain:
        - name: The original name of the file on the client machine.
        - type: The MIME type of the file (e.g., "image/jpeg").
        - size: The size of the file in bytes.
        - tmp_name: The temporary filename of the file in which the uploaded file was stored on the server.
        - error: The error code associated with this file upload (0 if no error).
    - **Move Temporary File:** The move_uploaded_file($_FILES["fileToUpload"]["tmp_name"], $destination) function is then used to securely move the uploaded file from its temporary location to its final destination on the server (e.g., uploads/). This is the only safe way to ensure the uploaded file is a legitimate upload and not a locally created file.

Two Security Considerations:

1. **File Type Validation:**
    - **Problem:** Malicious users might try to upload executable files (e.g., .php, .exe) disguised as images (e.g., malicious.php.jpg) or with genuine image extensions but containing malicious code. If these files are later accessed through the web server, they could execute and compromise the server.
    - **Solution:** Always validate the file type on the server-side.
        - **Check the file extension:** Use pathinfo() to get the actual extension and compare it against a whitelist of allowed extensions (e.g., "jpg", "png").

- **Check MIME type:** Although easily spoofed, checking the MIME type from $_FILES["fileToUpload"]["type"] adds another layer.
- **(Advanced) Use getimagesize() for images:** This function only works on actual image files and can confirm if the file is truly an image. If it returns false, it's not a valid image.

2. **File Size Validation:**
    - **Problem:** Uploading excessively large files can consume significant server resources (disk space, bandwidth, processing power), potentially leading to denial-of-service (DoS) attacks or simply slowing down the server.
    - **Solution:** Set and enforce a maximum file size limit on the server-side.
        - **PHP Configuration:** You can configure upload_max_filesize and post_max_size in your php.ini file to set overall limits.
        - **Script-level Check:** Use $_FILES["fileToUpload"]["size"] to check the file's size against a predefined maximum (e.g., 500KB) within your PHP script. If the file is too large, reject the upload and inform the user.

These checks, along with others like unique naming for uploaded files and storing them outside the web root if possible, are crucial for secure file uploads.

Q10. Decision-making program

PHP Program:

code PHP
downloadcontent_copy
expand_less

```
<?php
// Simulate a student's score (can be from form input in a real scenario)
$score = 72; // You can change this value to test different grades

echo "<h1>Student Grade Calculator</h1>";
echo "<p>Student Score: " . $score . "</p>";

// Decision-making using if-elseif-else
if ($score >= 90) {
    $grade = "A";
    echo "<p style='color: green;'>Grade: " . $grade . "</p>";
} elseif ($score >= 75) {
    $grade = "B";
    echo "<p style='color: blue;'>Grade: " . $grade . "</p>";
} elseif ($score >= 60) {
    $grade = "C";
    echo "<p style='color: orange;'>Grade: " . $grade . "</p>";
} else {
    $grade = "Fail";
    echo "<p style='color: red;'>Grade: " . $grade . "</p>";
}

echo "<hr>";

// Another example for demonstration
$anotherScore = 95;
echo "<p>Student Score: " . $anotherScore . "</p>";
```

```
if ($anotherScore >= 90) {
    echo "<p style='color: green;'>Grade: A</p>";
} elseif ($anotherScore >= 75) {
    echo "<p style='color: blue;'>Grade: B</p>";
} elseif ($anotherScore >= 60) {
    echo "<p style='color: orange;'>Grade: C</p>";
} else {
    echo "<p style='color: red;'>Grade: Fail</p>";
}
?>
```

Explanation of Decision-Making Statements:

Decision-making statements (also known as conditional statements or control flow statements) are fundamental programming constructs that allow a program to execute different blocks of code based on whether specified conditions evaluate to true or false. They enable programs to make choices and follow different paths of execution, leading to dynamic and responsive behavior.

How if-elseif-else Works:

- **if statement:** The most basic form. It executes a block of code *only if* its condition is true.
    - if (condition) { // code to execute if condition is true }
- **elseif statement:** Used after an if (or another elseif) to provide an alternative condition to test. If the preceding if condition was false, the elseif condition is evaluated. If it's true, its block of code is executed. You can have multiple elseif statements.
    - elseif (another_condition) { // code to execute if previous conditions were false and this one is true }
- **else statement:** The final, optional fallback. If none of the preceding if or elseif conditions evaluate to true, the code block within the else statement is executed.
    - else { // code to execute if all previous conditions were false }

In the provided program:

The program evaluates the $score variable sequentially:

1. It first checks if $score is 90 or greater. If true, it assigns 'A' and stops checking further conditions.
2. If the first condition is false, it then checks if $score is 75 or greater. If true, it assigns 'B' and stops.
3. If both previous conditions are false, it checks if $score is 60 or greater. If true, it assigns 'C' and stops.
4. If all above conditions are false, the else block is executed, assigning 'Fail'.

This structure ensures that only one block of code corresponding to the first true condition is executed, making it efficient for handling mutually exclusive choices.

Q11. Array types in PHP

PHP Script:

code PHP

downloadcontent_copy

expand_less

```php
<?php
echo "<h1>Demonstrating PHP Array Types</h1>";

// 1. Indexed Array
echo "<h2>1. Indexed Array</h2>";
// Declare an indexed array
$fruits = array("Apple", "Banana", "Cherry", "Date");
// Alternatively (short syntax): $fruits = ["Apple", "Banana", "Cherry",
"Date"];

echo "<p>Full Array: ";
print_r($fruits);
echo "</p>";

// Access an element by its numeric index (starting from 0)
echo "<p>Accessing the first fruit (index 0): " . $fruits[0] . "</p>"; //
Output: Apple
echo "<p>Accessing the third fruit (index 2): " . $fruits[2] . "</p>"; //
Output: Cherry

echo "<hr>";

// 2. Associative Array
echo "<h2>2. Associative Array</h2>";
// Declare an associative array (using named keys)
$person = array(
    "name"    => "John Doe",
    "age"     => 30,
    "city"    => "New York",
    "is_student" => false
);
// Alternatively (short syntax): $person = ["name" => "John Doe", "age" => 30];

echo "<p>Full Array: ";
print_r($person);
echo "</p>";

// Access an element by its named key
echo "<p>Accessing person's name: " . $person["name"] . "</p>"; // Output: John
Doe
echo "<p>Accessing person's age: " . $person["age"] . "</p>";   // Output: 30

echo "<hr>";

// 3. Multidimensional Array
echo "<h2>3. Multidimensional Array</h2>";
// Declare a multidimensional array (an array containing other arrays)
$students = array(
    array("Alice", 20, "Math"),      // Index 0: Alice's data (indexed sub-
array)
    array("Bob", 22, "Science"),     // Index 1: Bob's data
    "Charlie" => array("age" => 21, "major" => "Art") // Index "Charlie":
Charlie's data (associative sub-array)
);

echo "<p>Full Multidimensional Array: ";
print_r($students);
echo "</p>";
```

```
// Access elements from a multidimensional array
echo "<p>Accessing Alice's name: " . $students[0][0] . "</p>";      // Output:
Alice
echo "<p>Accessing Bob's major: " . $students[1][2] . "</p>";     // Output:
Science
echo "<p>Accessing Charlie's age: " . $students["Charlie"]["age"] . "</p>"; //
Output: 21
?>
```

Explanation of Array Types:

1. **Indexed Arrays:**
    - **Description:** Arrays where each element is associated with a numeric index, starting from 0 by default. PHP automatically assigns these integer indices.
    - **Declaration:** Elements are listed sequentially.
    - **Access:** Elements are accessed using their numeric index within square brackets (e.g., $array[0]).
    - **Use Case:** Ideal for storing a list of items where the order is important or elements are simply a collection without specific descriptive names.
2. **Associative Arrays:**
    - **Description:** Arrays where each element is associated with a named key (a string) instead of a numeric index. This allows you to give meaningful labels to your data.
    - **Declaration:** Keys are explicitly defined using the => operator (e.g., "key" => "value").
    - **Access:** Elements are accessed using their named key within square brackets (e.g., $array["name"]).
    - **Use Case:** Perfect for storing data records, like details of a person or a product, where each piece of information has a descriptive name.
3. **Multidimensional Arrays:**
    - **Description:** An array containing one or more other arrays. These can be arrays of indexed arrays, arrays of associative arrays, or a mix of both. They are used to store complex, tabular, or hierarchical data.
    - **Declaration:** Declared by nesting array definitions within each other.
    - **Access:** Elements are accessed using multiple sets of square brackets, corresponding to the level of nesting (e.g., $array[outer_key][inner_key]).
    - **Use Case:** Suitable for storing data like tables (rows and columns), matrices, lists of students with their details and grades, or complex configurations.

Q12. JSON parsing application

PHP Script:

code PHP
downloadcontent_copy

expand_less

```
<?php
echo "<h1>JSON Parsing Application (Encode & Decode)</h1>";

// 1. Create an associative array for a student
$student_info = array(
    "student_id" => "S001",
    "name"       => "Emily Blunt",
    "course"     => "Computer Science",
    "marks"      => array(
        "midterm" => 88,
        "final"   => 92
    ),
    "email"      => "emily.b@example.com"
);

echo "<h2>Original PHP Associative Array:</h2>";
echo "<pre>";
print_r($student_info);
echo "</pre>";

// 2. Convert it to JSON using json_encode()
$json_string = json_encode($student_info, JSON_PRETTY_PRINT);

echo "<h2>Converted JSON String:</h2>";
echo "<pre>";
echo $json_string;
echo "</pre>";

// 3. Decode it using json_decode()
// json_decode(json_string, true) decodes into an associative array
// json_decode(json_string) decodes into a standard object
$decoded_data = json_decode($json_string, true); // true for associative array

echo "<h2>Decoded PHP Value (Associative Array):</h2>";
echo "<pre>";
print_r($decoded_data);
echo "</pre>";

// 4. Display the student’s name and marks after decoding
if ($decoded_data !== null && is_array($decoded_data)) {
    echo "<h3>Displaying specific data after decoding:</h3>";
    echo "<p>Student Name: " . htmlspecialchars($decoded_data['name']) . "</p>";
    echo "<p>Midterm Marks: " . htmlspecialchars($decoded_data['marks']
['midterm']) . "</p>";
    echo "<p>Final Marks: " . htmlspecialchars($decoded_data['marks']
['final']) . "</p>";
} else {
    echo "<p style='color: red;'>Error decoding JSON or data is not an
array.</p>";
}

// Example decoding to an object (default behavior of json_decode without
'true')
$decoded_object = json_decode($json_string);
echo "<h2>Decoded PHP Value (Standard Object):</h2>";
echo "<pre>";
print_r($decoded_object);
echo "</pre>";
if ($decoded_object !== null && is_object($decoded_object)) {
    echo "<h3>Displaying specific data from object:</h3>";
    echo "<p>Student Name (from object): " . htmlspecialchars($decoded_object-
>name) . "</p>";
```

```
    echo "<p>Midterm Marks (from object): " . htmlspecialchars($decoded_object->marks->midterm) . "</p>";
}
?>
```

Part-3: Files, Cookies, Sessions, and Validation

Q13. include vs require

header.php:

code PHP
downloadcontent_copy
expand_less

```
<?php
// header.php
echo "<!DOCTYPE html>";
echo "<html>";
echo "<head><title>My PHP Page</title></head>";
echo "<body>";
echo "<h1>Welcome to My College Portal</h1>";
echo "<nav>";
echo "  <a href='#'>Home</a> | ";
echo "  <a href='#'>About Us</a> | ";
echo "  <a href='#'>Contact</a>";
echo "</nav>";
echo "<hr>";
?>
```

main_include.php:

code PHP
downloadcontent_copy
expand_less

```
<?php
// main_include.php
include "header.php"; // Includes the header.php file

echo "<h2>This is the main content area (using include)</h2>";
echo "<p>Here's some information about our services.</p>";

// Attempt to include a non-existent file
include "non_existent_file.php";
echo "<p style='color: orange;'>This line will still execute even if 'non_existent_file.php' failed to include (due to include).</p>";

echo "<hr>";
echo "<p>Footer content.</p>";
echo "</body>";
echo "</html>";
?>
```

main_require.php:

code PHP
downloadcontent_copy
expand_less

```php
<?php
// main_require.php
require "header.php"; // Requires the header.php file

echo "<h2>This is the main content area (using require)</h2>";
echo "<p>Here's some information about our services.</p>";

// Attempt to include a non-existent file
require "another_non_existent_file.php"; // This will cause a fatal error and
stop script execution
echo "<p style='color: red;'>This line will NOT execute if
'another_non_existent_file.php' failed to require.</p>"; // This line will never
be reached

echo "<hr>";
echo "<p>Footer content.</p>";
echo "</body>";
echo "</html>";
?>
```

To run these:

1. Save the files as header.php, main_include.php, and main_require.php in your web server's document root.
2. Navigate to main_include.php in your browser. You will see the header, main content, and a warning about the missing file, but the script will continue.
3. Navigate to main_require.php in your browser. You will see the header, main content, but then a fatal error, and the script will terminate immediately, preventing the footer from being displayed.

Explanation of the Difference in Error Handling:

Both include and require are used to embed the content of one PHP file into another. However, their behavior when the specified file cannot be found or loaded differs significantly in terms of error handling:

1. **include:**
    - **Error Level:** If the include statement fails to find or load the specified file, it generates a **E_WARNING** error.
    - **Script Execution:** Despite the warning, the PHP script will **continue its execution**. This means any code following the include statement will still be processed.
    - **Use Case:** include is often used for non-critical files, like optional templates, secondary content blocks, or shared functions that aren't absolutely essential for the script's core functionality. If the included file is missing, it's not a catastrophic failure.
2. **require:**
    - **Error Level:** If the require statement fails to find or load the specified file, it generates a **E_COMPILE_ERROR** (a fatal error).

- **Script Execution:** Upon encountering a fatal error, PHP will **immediately terminate script execution**. No further code will be processed or outputted after the require statement fails.
- **Use Case:** require is typically used for critical files that are essential for the proper functioning of the script, such as configuration files, database connection scripts, or core libraries. If these files are missing, the application cannot proceed, and it's better to halt execution to prevent further issues.

In summary, require is stricter than include. If the included file is absolutely necessary for the script to run, use require. If the script should continue even if the file is missing, use include.

Q14. Cookies in web apps

PHP Script (set_cookie.php):

code PHP
downloadcontent_copy
expand_less

```
<?php
// set_cookie.php
$cookie_name = "username";
$cookie_value = "Ravi";
$expiry_time = time() + (86400 * 1); // 86400 seconds = 1 day

// Set the cookie
setcookie($cookie_name, $cookie_value, $expiry_time, "/"); // "/" makes it
available throughout the domain

echo "<h1>Cookie Set!</h1>";
echo "<p>Cookie '" . $cookie_name . "' with value '" . $cookie_value . "' has
been set. It will expire in 1 day.</p>";
echo "<p>You can check your browser's developer tools (Application/Storage tab)
to see the cookie.</p>";

// You might redirect the user or provide a link to a page that reads the cookie
echo "<p><a href='read_cookie.php'>Go to a page that reads the cookie</a></p>";
?>
```

read_cookie.php:

code PHP
downloadcontent_copy
expand_less

```
<?php
// read_cookie.php
echo "<h1>Reading Cookie</h1>";

if (isset($_COOKIE["username"])) {
    echo "<p>Welcome back, " . htmlspecialchars($_COOKIE["username"]) . "!</p>";
} else {
    echo "<p>No 'username' cookie found.</p>";
    echo "<p>Please visit <a href='set_cookie.php'>set_cookie.php</a> first to
set the cookie.</p>";
}
?>
```

Explanation of Two Uses of Cookies:

Cookies are small pieces of data that a server sends to a user's web browser. The browser stores these cookies and sends them back to the server with subsequent requests to the same domain. They are primarily used to remember stateful information in the stateless HTTP protocol.

1. **Remembering User Preferences/Personalization:**
    - **Description:** Cookies are commonly used to store user-specific settings and preferences, allowing websites to deliver a personalized experience.
    - **Example:** Storing a user's preferred language, theme (dark/light mode), currency, or layout choices. When the user revisits the site, the cookie is sent, and the server can retrieve these preferences to present the website in the user's desired configuration without requiring them to re-select options.
2. **Session Management (e.g., User Login Status):**
    - **Description:** Cookies are fundamental for maintaining a user's logged-in state across multiple page requests. Since HTTP is stateless, the server needs a way to identify a user who has authenticated.
    - **Example:** After a user successfully logs in, the server generates a unique session ID and stores it in a cookie (often named PHPSESSID for PHP). This cookie is sent with every subsequent request. The server uses this session ID to retrieve the user's session data (which might contain their username, roles, etc., stored on the server), effectively keeping the user "logged in" as they navigate the site. Without cookies for session IDs, users would have to log in on every single page load.

Q15. File existence check

PHP Script:

code PHP
downloadcontent_copy
expand_less

```
<?php
echo "<h1>File Existence Check</h1>";

$filename = "data.txt";
$nonExistentFile = "non_existent.txt";

// 1. Create data.txt for demonstration
if (!file_exists($filename)) {
    file_put_contents($filename, "This is the first line.\nThis is the second
line.");
    echo "<p>Created " . $filename . " for demonstration purposes.</p>";
}

// Check if data.txt exists
if (file_exists($filename)) {
    echo "<p style='color: green;'>'" . $filename . "' found.</p>";
} else {
    echo "<p style='color: red;'>'" . $filename . "' not found.</p>";
}

echo "<br>";

// Check for a non-existent file
```

```
if (file_exists($nonExistentFile)) {
    echo "<p style='color: green;'>'" . $nonExistentFile . "' found.</p>";
} else {
    echo "<p style='color: red;'>'" . $nonExistentFile . "' not found.</p>";
}
?>
```

Explanation of the Function Used:

The function used to check if a file or directory exists in PHP is file_exists().

Purpose of file_exists():
The file_exists() function checks whether a file or directory specified by the given filename exists. It returns TRUE if the file or directory exists, and FALSE otherwise.

How it works:

- It takes a single argument: the path to the file or directory you want to check.
- The path can be absolute (e.g., /var/www/html/data.txt) or relative to the current script's execution directory (e.g., data.txt).
- It's a crucial function for file and directory operations, as it allows you to prevent errors (like trying to open a non-existent file for reading) and implement conditional logic based on the presence of resources. For instance, you might use it to create a directory only if it doesn't already exist, or to

validate if a user-uploaded file (like a profile picture) already exists under a certain name.

Q16. Reading single line from file

PHP Program:

code PHP
downloadcontent_copy
expand_less

```
<?php
echo "<h1>Reading First Line from File</h1>";

$filename = "data.txt";

// Ensure data.txt exists with at least one line for demonstration
if (!file_exists($filename)) {
    file_put_contents($filename, "This is the first line of the file.\nThis is
the second line.\nAnd this is the third line.");
    echo "<p>Created '" . $filename . "' with content for demonstration.</p>";
}

// Check if the file exists before attempting to open
if (file_exists($filename)) {
    // Open the file in read mode ('r')
    $file_handle = fopen($filename, "r");

    if ($file_handle) {
        // Read the first line using fgets()
        $first_line = fgets($file_handle);
```

```
        echo "<p>The first line from '" . $filename . "' is:</p>";
        echo "<p style='font-weight: bold; color: blue;'>" .
htmlspecialchars($first_line) . "</p>";

        // Close the file handle
        fclose($file_handle);
    } else {
        echo "<p style='color: red;'>Error: Could not open the file '" .
$filename . "'.</p>";
    }
} else {
    echo "<p style='color: red;'>Error: The file '" . $filename . "' does not
exist.</p>";
}
?>
```

Explanation of the Function Used:

The function used to read a single line from an open file in PHP is fgets().

Purpose of fgets():
The fgets() function reads a line from a file pointer (resource) until it reaches either the end of the line (indicated by a newline character), the end of the file (EOF), or the specified maximum length. The newline character is included in the returned string.

How it works:

1. **fopen($filename, "r")**: First, fopen() is used to open the file. The "r" mode specifies that the file should be opened for reading only. If the file is successfully opened, fopen() returns a file pointer (a resource), which is then passed to fgets().
2. **fgets($file_handle)**: This function reads data from the file pointer ($file_handle). It starts reading from the current position of the file pointer and stops when it encounters a newline character (\n), the end of the file, or a buffer limit.
3. **File Pointer Movement:** After fgets() reads a line, the file pointer automatically moves to the beginning of the next line. This allows you to call fgets() repeatedly in a loop to read the entire file line by line.
4. **fclose($file_handle)**: After all read operations are complete, it's crucial to close the file handle using fclose() to free up system resources.

In the example, fgets($file_handle) is called only once, ensuring that only the very first line of data.txt is read and displayed.

Q17. htmlspecialchars() in forms

PHP Form (input_form.php):

code PHP
downloadcontent_copy
expand_less

```
<!DOCTYPE html>
<html>
<head>
    <title>Input Form with htmlspecialchars()</title>
</head>
```

```
<body>
    <h1>Enter Your Comment</h1>
    <form action="input_form.php" method="POST">
        <label for="comment">Your Comment:</label><br>
        <textarea id="comment" name="comment" rows="5" cols="40"><?php
            // Pre-fill the textarea with the submitted (and protected) comment
            if (isset($_POST['comment'])) {
                echo htmlspecialchars($_POST['comment']);
            }
        ?></textarea><br><br>
        <input type="submit" value="Submit Comment">
    </form>

    <?php
    if (isset($_POST['comment'])) {
        $raw_comment = $_POST['comment'];
        $safe_comment = htmlspecialchars($raw_comment);

        echo "<h2>Your Comment (Raw Output - DANGEROUS!):</h2>";
        // DO NOT DO THIS IN PRODUCTION! This shows the vulnerability.
        echo "<div style='border: 1px solid red; padding: 10px; margin-bottom:
20px;'>" . $raw_comment . "</div>";
        echo "<p style='color: red;'>If you typed HTML/JavaScript, it might
execute above!</p>";


        echo "<h2>Your Comment (Safe Output with htmlspecialchars()):</h2>";
        echo "<div style='border: 1px solid green; padding: 10px;'>" .
$safe_comment . "</div>";
        echo "<p style='color: green;'>HTML/JavaScript tags are converted to
entities and will display as plain text.</p>";
    }
    ?>
</body>
</html>
```

How to Test the Vulnerability:

In the "Your Comment" textarea, try entering something like:

Hello, <script>alert('XSS Attack!');</script> This is a **bold** comment.

or

Hey! <img src="nonexistent.png" onerror="alert('Image load failed and XSS executed!');">

Explanation of why this function is important:

The htmlspecialchars() function is critically important in web development, especially when displaying user-submitted data back to a web page. Its primary purpose is to **convert special HTML characters into HTML entities**.

Specifically, it converts:

- & (ampersand) to &
- " (double quote) to " (when ENT_NOQUOTES is not set)
- ' (single quote) to ' (or ' in HTML5, when ENT_QUOTES is set)
- < (less than) to <
- > (greater than) to >

Why this is important (Preventing HTML/Script Injection / Cross-Site Scripting - XSS):

The main reason htmlspecialchars() is vital is to prevent **Cross-Site Scripting (XSS)** attacks.

- **XSS Explained:** XSS is a type of security vulnerability where attackers inject malicious client-side scripts (often JavaScript) into web pages viewed by other users. If a website takes user input (like a comment, forum post, or profile description) and displays it directly back to the browser without proper sanitization, an attacker can embed <script> tags or other HTML elements with malicious code.
- **The Attack:** When another user views that page, their browser will execute the injected script. This script could then:
    - Steal their session cookies (allowing the attacker to hijack their login).
    - Deface the website.
    - Redirect users to malicious sites.
    - Phish for credentials.
    - Perform actions on behalf of the user.
- **How htmlspecialchars() Helps:** By converting characters like <, >, &, and " into their HTML entity equivalents, htmlspecialchars() ensures that these characters are treated as plain text by the browser, not as actual HTML or JavaScript code.
    - For example, if a user submits <script>alert('hello');</script>, htmlspecialchars() will convert it to <script>alert('hello');</script>.
    - When the browser renders this, it will literally display <script>alert('hello');</script> on the page, rather than executing the JavaScript alert().

Therefore, htmlspecialchars() is a crucial first line of defense against XSS attacks, safeguarding both your website and its users. It should be used any time user-supplied data is outputted to an HTML page.

Q18. Directory functions

PHP Script:

code PHP
downloadcontent_copy
expand_less

```
<?php
echo "<h1>Directory Management</h1>";

$dir_name = "testdir";

// 1. Create a directory named testdir
echo "<h2>Creating Directory: '" . $dir_name . "'</h2>";
if (!is_dir($dir_name)) { // Check if directory already exists
    if (mkdir($dir_name, 0777, true)) { // 0777 grants full permissions, true
allows recursive creation
        echo "<p style='color: green;'>Directory '" . $dir_name . "' created
successfully.</p>";
    } else {
        echo "<p style='color: red;'>Failed to create directory '" . $dir_name .
"'.</p>";
    }
```

```
} else {
    echo "<p style='color: orange;'>Directory '" . $dir_name . "' already
exists.</p>";
}

// Optional: Create a dummy file inside for rm_dir to show it must be empty
file_put_contents($dir_name . "/dummy.txt", "This is a dummy file.");
echo "<p>Created a dummy file inside '" . $dir_name . "' to demonstrate rmdir
requires empty directory.</p>";


// 2. Remove it after creation
echo "<h2>Removing Directory: '" . $dir_name . "'</h2>";
if (is_dir($dir_name)) {
    // Attempt to remove with content (will fail)
    if (rmdir($dir_name)) {
        echo "<p style='color: green;'>Directory '" . $dir_name . "' removed
successfully.</p>";
    } else {
        echo "<p style='color: orange;'>Failed to remove directory '" .
$dir_name . "'. (It must be empty first)</p>";
        // Remove dummy file first
        if (file_exists($dir_name . "/dummy.txt")) {
            unlink($dir_name . "/dummy.txt");
            echo "<p>Removed dummy.txt from '" . $dir_name . "'.</p>";
        }
        // Try removing again after emptying
        if (rmdir($dir_name)) {
            echo "<p style='color: green;'>Directory '" . $dir_name . "' removed
successfully after emptying.</p>";
        } else {
            echo "<p style='color: red;'>Still failed to remove directory '" .
$dir_name . "'.</p>";
        }
    }
} else {
    echo "<p style='color: orange;'>Directory '" . $dir_name . "' does not exist
to remove.</p>";
}
?>
```

Explanation of Two Directory Functions Used:

1. **mkdir() (Make Directory):**
    - **Purpose:** The mkdir() function is used to create a new directory (folder) on the file system.
    - **Syntax:** mkdir(string $pathname, int $permissions = 0777, bool $recursive = false)
        - $pathname: The path of the directory to be created.
        - $permissions (optional): The permissions for the new directory (e.g., 0777 for full read/write/execute for everyone, 0755 for owner full, group/others read/execute).
        - $recursive (optional): If true, allows nested directories to be created if parent directories don't exist.

- **How it was used:** mkdir($dir_name, 0777, true) attempts to create testdir. The true argument is important if testdir might be part of a path like parent/testdir and parent doesn't exist.

2. **rmdir() (Remove Directory):**
    - **Purpose:** The rmdir() function is used to delete an existing empty directory.
    - **Syntax:** rmdir(string $dirname)
        - $dirname: The path of the directory to be removed.
    - **Important Note:** rmdir() can **only remove empty directories**. If the directory contains any files or subdirectories, rmdir() will fail and return false, often with a warning. To remove non-empty directories, you typically need to recursively delete all its contents first before calling rmdir() on the empty directory itself.
    - **How it was used:** rmdir($dir_name) attempts to delete testdir. The script demonstrates that it will fail if dummy.txt is present and succeeds only after dummy.txt is unlink()ed (deleted).

Q19. File open modes

PHP Program:

code PHP
downloadcontent_copy
expand_less

```
<?php
echo "<h1>File Write with 'w' Mode</h1>";

$filename = "write_test.txt";
$content_to_write = "This is a new line written in 'w' mode.\n";
$another_line = "This line will overwrite the previous content if 'w' is used
again.\n";

// Scenario 1: File doesn't exist or is empty
echo "<h2>Scenario 1: File " . $filename . " initially does not exist or is
empty</h2>";
// Ensure the file is fresh for this scenario
if (file_exists($filename)) {
    unlink($filename); // Delete previous file if any
    echo "<p>Deleted existing '" . $filename . "' to start fresh.</p>";
}

// Open file using "w" mode and write a line
$file_handle_w = fopen($filename, "w");
if ($file_handle_w) {
    fwrite($file_handle_w, $content_to_write);
    fclose($file_handle_w);
    echo "<p style='color: green;'>Successfully wrote: '" .
htmlspecialchars($content_to_write) . "' to '" . $filename . "' in 'w'
mode.</p>";
    echo "<p>Current content of '" . $filename . "': <pre style='background-
color: #f0f0f0; padding: 5px;'>" .
htmlspecialchars(file_get_contents($filename)) . "</pre></p>";
} else {
    echo "<p style='color: red;'>Failed to open '" . $filename . "' in 'w' mode
for writing.</p>";
```

```
}

echo "<hr>";

// Scenario 2: File already exists with content
echo "<h2>Scenario 2: File " . $filename . " already exists with content</h2>";
echo "<p>Content of '" . $filename . "' before next write: <pre
style='background-color: #f0f0f0; padding: 5px;'>" .
htmlspecialchars(file_get_contents($filename)) . "</pre></p>";

// Open the same file again using "w" mode and write another line
$file_handle_w2 = fopen($filename, "w"); // This will truncate the file!
if ($file_handle_w2) {
    fwrite($file_handle_w2, $another_line);
    fclose($file_handle_w2);
    echo "<p style='color: green;'>Successfully wrote: '" .
htmlspecialchars($another_line) . "' to '" . $filename . "' in 'w' mode (this
time it truncated existing content).</p>";
    echo "<p>Current content of '" . $filename . "': <pre style='background-
color: #f0f0f0; padding: 5px;'>" .
htmlspecialchars(file_get_contents($filename)) . "</pre></p>";
} else {
    echo "<p style='color: red;'>Failed to open '" . $filename . "' in 'w' mode
for writing (second attempt).</p>";
}
?>
```

Explanation of what "w" mode does:

The "w" (write) mode in PHP's fopen() function is used to open a file for writing only. It has specific and very important behaviors:

1. **File Creation:** If the specified file does not exist, fopen() in "w" mode will attempt to **create the file**.
2. **File Truncation (Overwrite):** If the file **does exist**, fopen() in "w" mode will **truncate its size to zero bytes**. This means all existing content in the file will be deleted, and the file becomes empty. Any subsequent writes will start from the beginning of this now-empty file.
3. **Pointer Position:** The file pointer is placed at the **beginning of the file**.
4. **Write Only:** You can only write data to the file in this mode; reading operations are not allowed.

In summary: "w" mode is used when you want to create a new file or completely overwrite the content of an existing file. If you want to add content to the end of an existing file without deleting its current content, you would use the "a" (append) mode instead.

Part-4: Superglobals, Cookies, Directories, Validation, Strings

Q20. $_SERVER and $_SESSION

PHP Script (server_session.php):

code PHP
downloadcontent_copy
expand_less

```php
<?php
// Start the session at the very beginning of the script
session_start();

echo "<h1>Demonstrating \$_SERVER and \$_SESSION</h1>";

// 1. Display server name using $_SERVER
echo "<h2>Using \$_SERVER Superglobal:</h2>";
if (isset($_SERVER['SERVER_NAME'])) {
    echo "<p>Server Name: " . htmlspecialchars($_SERVER['SERVER_NAME']) .
"</p>";
} else {
    echo "<p>Server Name not available.</p>";
}

if (isset($_SERVER['REQUEST_METHOD'])) {
    echo "<p>Request Method: " . htmlspecialchars($_SERVER['REQUEST_METHOD']) .
"</p>";
}

if (isset($_SERVER['PHP_SELF'])) {
    echo "<p>Current Script Path: " . htmlspecialchars($_SERVER['PHP_SELF']) .
"</p>";
}

echo "<hr>";

// 2. Start a session and store username
echo "<h2>Using \$_SESSION Superglobal:</h2>";
$session_username = "John_Doe"; // Example username to store

if (!isset($_SESSION['user_id'])) {
    // If user_id is not set, meaning it's a new session or user not logged in
    $_SESSION['user_id'] = uniqid(); // Generate a unique ID for the session
    $_SESSION['username'] = $session_username;
    $_SESSION['login_time'] = date('Y-m-d H:i:s');
    echo "<p style='color: green;'>Session started and username '" .
htmlspecialchars($session_username) . "' stored.</p>";
} else {
    // If session already exists
    echo "<p style='color: blue;'>Session already active.</p>";
    echo "<p>Retrieved Username: " . htmlspecialchars($_SESSION['username']) .
"</p>";
    echo "<p>Logged In Since: " . htmlspecialchars($_SESSION['login_time']) .
"</p>";
}

echo "<p>Your current session ID: " . session_id() . "</p>";
echo "<p><a href='server_session.php'>Refresh this page</a> to see session data
persist.</p>";
echo "<p><a href='logout.php'>Logout (destroy session)</a></p>";
?>
```

logout.php (for session demonstration):

code PHP

downloadcontent_copy

expand_less

```php
<?php
session_start(); // Start the session to access session variables

// Unset all session variables
```

```
$_SESSION = array();

// If it's desired to kill the session, also delete the session cookie.
// Note: This will destroy the session, and not just the session data!
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}

// Finally, destroy the session.
session_destroy();

echo "<h1>Logged Out</h1>";
echo "<p>You have been successfully logged out.</p>";
echo "<p><a href='server_session.php'>Go back to the main page</a></p>";
?>
```

Explanation of the Purpose of Both Superglobals:

1. **$_SERVER Superglobal:**
    - **Purpose:** The $_SERVER superglobal is an associative array that contains information about the server and execution environment, such as headers, paths, and script locations. It's automatically populated by the web server and PHP itself.
    - **Key Information Provided:**
        - $_SERVER['SERVER_NAME']: The name of the host server (e.g., localhost, www.example.com).
        - $_SERVER['REQUEST_METHOD']: The request method used to access the page (e.g., GET, POST, PUT).
        - $_SERVER['PHP_SELF']: The filename of the currently executing script, relative to the document root. Useful for form action attributes.
        - $_SERVER['REMOTE_ADDR']: The IP address from which the user is viewing the current page.
        - $_SERVER['HTTP_USER_AGENT']: The User-Agent string from the current request (browser and OS information).
    - **Use Cases:** Useful for debugging, logging, routing requests, form validation ($_SERVER["REQUEST_METHOD"]), and determining server configurations.
2. **$_SESSION Superglobal:**
    - **Purpose:** The $_SESSION superglobal is an associative array used to store session variables. A session provides a way to store information (in variables) to be used across multiple pages and requests by a single user. Unlike cookies, session data is stored on the server, and only a unique session ID (often stored in a cookie) is sent to the client.
    - **Key Behavior:**

        - session_start(): Must be called at the very beginning of a script to start or resume a session.
        - **Persistence:** Data stored in $_SESSION persists for the duration of the user's session (until the browser is closed or the session expires/is destroyed explicitly).
        - **User-Specific:** Each user gets their own unique session.
    - **Use Cases:** Essential for implementing:
        - **User Authentication:** Keeping users logged in as they navigate a website.
        - **Shopping Carts:** Storing items a user adds to their cart.
        - **User Preferences:** Storing preferences that need to persist across pages but shouldn't be directly stored on the client (like in cookies).
        - **Temporary Data:** Passing data between pages without using GET or POST parameters (e.g., a multi-step form).

Q21. Set cookie for preferred language

PHP Script (set_language_cookie.php):

code PHP
downloadcontent_copy
expand_less

```
<?php
// Define cookie parameters
$cookie_name = "language";
$cookie_value = "English";
$expiry_time = time() + (86400 * 30); // 86400 seconds = 1 day, so 30 days
$path = "/"; // Cookie available throughout the entire domain
$domain = ""; // Empty for current domain, or specify "example.com"
$secure = false; // Set to true if using HTTPS
$httponly = true; // Set to true to prevent JavaScript access (good for
security)

// Set the cookie
setcookie($cookie_name, $cookie_value, $expiry_time, $path, $domain, $secure,
$httponly);

echo "<h1>Language Cookie Set!</h1>";
echo "<p>Cookie '" . htmlspecialchars($cookie_name) . "' with value '" .
htmlspecialchars($cookie_value) . "' has been set.</p>";
echo "<p>It will expire in 30 days.</p>";
echo "<p>Refresh or visit <a
href='read_language_cookie.php'>read_language_cookie.php</a> to see it in
action.</p>";
?>
```

read_language_cookie.php (to demonstrate reading the cookie):

code PHP
downloadcontent_copy
expand_less

```
<?php
echo "<h1>Reading Language Cookie</h1>";
```

```
if (isset($_COOKIE["language"])) {
    echo "<p>Your preferred language is: <strong style='color: blue;'>" .
htmlspecialchars($_COOKIE["language"]) . "</strong></p>";
} else {
    echo "<p style='color: orange;'>No 'language' cookie found. Defaulting to
English.</p>";
    echo "<p>Visit <a href='set_language_cookie.php'>set_language_cookie.php</a>
to set it.</p>";
}
?>
```

Explanation of Syntax of Setting Cookies:

The setcookie() function in PHP is used to send an HTTP cookie to the client (user's browser). It must be called *before* any actual HTML output or other HTTP headers are sent to the browser.

The full syntax for setcookie() is:

setcookie(name, value, expire, path, domain, secure, httponly);

Let's break down each parameter:

1. **name (Required):**
    - The name of the cookie. This is how you'll access its value later (e.g., $_COOKIE['language']).
2. **value (Optional):**
    - The value to be stored in the cookie. This value is stored on the client's machine.
3. **expire (Optional):**
    - The time when the cookie will expire. This is a Unix timestamp (seconds since the Unix Epoch). If this parameter is omitted or set to 0, the cookie will expire when the browser is closed (a "session cookie").
    - **Common usage:** time() + (seconds_until_expiry). For 30 days: time() + (86400 * 30).
4. **path (Optional):**
    - The path on the server where the cookie will be available.
    - A single slash (/) means the cookie is available throughout the entire domain (i.e., all paths).
    - /my_app/ means it's only available within the /my_app/ directory and its subdirectories.
5. **domain (Optional):**
    - The domain that the cookie is available to.
    - To make a cookie available to all subdomains of example.com (e.g., www.example.com, blog.example.com), you would set the domain to .example.com (note the leading dot).

- Leaving it empty usually defaults to the current host, excluding subdomains.

6. **secure (Optional):**
    - A boolean value (true or false). If true, the cookie will only be sent over secure HTTPS connections. If false, it can be sent over both HTTP and HTTPS.
    - **Recommendation:** Always set this to true for sensitive cookies in production environments.
7. **httponly (Optional):**
    - A boolean value (true or false). If true, the cookie will be made inaccessible to client-side scripting languages like JavaScript. This is a crucial security measure to mitigate Cross-Site Scripting (XSS) attacks by preventing malicious scripts from accessing the cookie.
    - **Recommendation:** Always set this to true for all non-JavaScript-dependent cookies (especially session IDs).

Q22. Directory parsing

PHP Script:

code PHP

downloadcontent_copy

expand_less

```
<?php
echo "<h1>Directory Content Lister</h1>";

$directory = "./"; // Current directory. Change this to a specific folder like
"uploads/"
$test_directory = "my_test_folder";

// Create a test directory and some dummy files for demonstration
if (!is_dir($test_directory)) {
    mkdir($test_directory, 0777, true);
    file_put_contents($test_directory . "/file1.txt", "Content of file 1");
    file_put_contents($test_directory . "/image.jpg", "Dummy image content");
    mkdir($test_directory . "/subfolder");
    file_put_contents($test_directory . "/subfolder/subfile.html",
"<html></html>");
    echo "<p>Created '" . $test_directory . "' and some dummy files/subfolders
for demo.</p>";
}

echo "<h2>Listing contents of directory: '" .
htmlspecialchars($test_directory) . "'</h2>";

// Open the directory
if ($handle = opendir($test_directory)) {
    echo "<ul>";
    // Read all files inside it and display their names using a loop
    while (false !== ($entry = readdir($handle))) {
        // Exclude "." and ".." which represent the current and parent
directories
        if ($entry != "." && $entry != "..") {
            echo "<li>" . htmlspecialchars($entry) . "</li>";
        }
    }
```

```
    echo "</ul>";
    // Close the directory handle
    closedir($handle);
} else {
    echo "<p style='color: red;'>Error: Could not open directory '" .
htmlspecialchars($test_directory) . "'.</p>";
}

// Clean up the test directory and its contents after demonstration (optional)
// To remove, you'd need a recursive delete function or manual cleanup
/*
if (is_dir($test_directory)) {
    unlink($test_directory . "/file1.txt");
    unlink($test_directory . "/image.jpg");
    unlink($test_directory . "/subfolder/subfile.html");
    rmdir($test_directory . "/subfolder");
    rmdir($test_directory);
    echo "<p>Cleaned up '" . $test_directory . "'.</p>";
}
*/
?>
```

Explanation of Directory Parsing:

Directory parsing (or listing directory contents) involves programmatically accessing a directory on the file system and iterating through its entries (files and subdirectories). PHP provides a set of functions for this purpose, primarily involving opendir(), readdir(), and closedir().

How the script works:

1. **opendir($directory):**
    - This function is used to open a directory handle. It takes the path to the directory as an argument.
    - If successful, it returns a directory handle (a resource) that can be used by other directory functions. If it fails (e.g., directory doesn't exist or permissions issues), it returns false.
    - In the script, opendir($test_directory) opens the "my_test_folder".
2. **readdir($handle):**
    - This function reads the next entry from the directory stream specified by the directory handle.
    - It returns the filename of the next entry, or false if there are no more entries or an error occurs.
    - The while (false !== ($entry = readdir($handle))) loop is a common pattern:
        - It continuously calls readdir() to get each file/directory name ($entry) one by one.
        - The false !== comparison is crucial because a valid filename could potentially be 0 (which is loosely equal to false), so a strict comparison prevents unintended loop termination.

- Inside the loop, it checks for . (current directory) and .. (parent directory) entries, which are standard in file systems but usually not what you want to list to the user, and skips them.

3. **closedir($handle):**
    - After you have finished reading all entries or no longer need to access the directory, closedir() is used to close the directory handle. This frees up system resources associated with the opened directory.

By combining these functions, the script effectively opens my_test_folder, iterates through all its contents, and displays the name of each file and subfolder within it.

Q23. Form validation (Important)

PHP Form (validation_form.php):

code PHP
downloadcontent_copy
expand_less

```
<!DOCTYPE html>
<html>
<head>
    <title>User Registration</title>
    <style>
        .error { color: red; font-size: 0.9em; }
        .success { color: green; font-weight: bold; }
        label { display: block; margin-top: 10px; }
        input[type="text"], input[type="email"] { width: 300px; padding: 5px; }
        input[type="submit"] { margin-top: 15px; padding: 8px 15px; cursor:
pointer; }
    </style>
</head>
<body>
    <h1>Register New User</h1>

    <?php
    $username = $email = "";
    $username_error = $email_error = "";
    $form_valid = true;

    if ($_SERVER["REQUEST_METHOD"] == "POST") {
        // Validate Username
        if (empty($_POST["username"])) {
            $username_error = "Username is required.";
            $form_valid = false;
        } else {
            $username = trim($_POST["username"]);
            // Check if username contains only letters and numbers
            if (!preg_match("/^[a-zA-Z0-9]*$/", $username)) {
                $username_error = "Username can only contain letters and
numbers.";
                $form_valid = false;
            }
        }

        // Validate Email
        if (empty($_POST["email"])) {
            $email_error = "Email is required.";
            $form_valid = false;
```

```
        } else {
            $email = trim($_POST["email"]);
            // Check if email format is valid
            if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
                $email_error = "Invalid email format.";
                $form_valid = false;
            }
        }

        // If all validations pass
        if ($form_valid) {
            echo "<p class='success'>Registration successful for " .
htmlspecialchars($username) . " (" . htmlspecialchars($email) . ")!</p>";
            // In a real application, you would save data to a database,
            // send a confirmation email, redirect, etc.
            // Reset fields after successful submission
            $username = $email = "";
        } else {
            echo "<p class='error'>Please correct the errors below.</p>";
        }
    }
    ?>

    <form action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>"
method="POST">
        <label for="username">Username:</label>
        <input type="text" id="username" name="username" value="<?php echo
htmlspecialchars($username); ?>">
        <span class="error"><?php echo $username_error; ?></span>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" value="<?php echo
htmlspecialchars($email); ?>">
        <span class="error"><?php echo $email_error; ?></span>

        <input type="submit" value="Register">
    </form>
</body>
</html>
```

Explanation of Importance of Form Validation:

Form validation is the process of ensuring that user-submitted data through a web form meets specific requirements and is in the correct format before it is processed by the server or stored in a database. It is a critical aspect of web development for several important reasons:

1. **Security:** This is the most crucial reason. Unvalidated or poorly validated user input is a major vector for various web vulnerabilities:
    - **SQL Injection:** Malicious SQL code injected into form fields can compromise databases.
    - **Cross-Site Scripting (XSS):** Malicious scripts injected into input fields can execute on other users' browsers (as demonstrated with htmlspecialchars()).
    - **Command Injection:** In some cases, OS commands can be injected.
    - **File Path Traversal:** Attackers could try to access unauthorized files on the server. Validation helps sanitize and escape input, mitigating these risks.

2. **Data Integrity:** Ensures that only correct, complete, and properly formatted data enters your application's backend and database. This prevents broken records, inconsistent data, and future errors when retrieving or processing the data. For example, ensuring an email address is actually an email and not just random text.
3. **User Experience (UX):**
    - **Feedback:** Provides immediate and clear feedback to users when they make mistakes in filling out a form, guiding them to correct their input.
    - **Prevention of Frustration:** Prevents users from submitting a form only to find out later that it failed due to incorrect data, forcing them to redo large parts of it.
    - **Guidance:** Helps users understand what kind of input is expected in each field.
4. **Resource Efficiency:** Prevents the server from processing invalid data, reducing unnecessary load on the server, database, and network resources. It's more efficient to catch errors early.

Validation in the provided script:

- **Username Validation:**
    - empty($_POST["username"]): Checks if the username field is empty.
    - !preg_match("/^[a-zA-Z0-9]*$/", $username): Uses a regular expression to ensure the username contains only alphanumeric characters (letters and numbers).
- **Email Validation:**
    - empty($_POST["email"]): Checks if the email field is empty.
    - !filter_var($email, FILTER_VALIDATE_EMAIL): Uses PHP's built-in filter_var() function with FILTER_VALIDATE_EMAIL to check if the email address conforms to a valid email format. This is generally more robust than custom regular expressions for email.
- **htmlspecialchars($_SERVER["PHP_SELF"])**: Ensures that the action attribute of the form is safe from XSS attacks, pointing to the current script securely.
- **htmlspecialchars($username) / htmlspecialchars($email)**: Used when displaying the user's input back